GUIA TÉCNICO | 02

Tensores, TensorFlow e Keras

Capítulo 02/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Reconhecer shapes, dtype, broadcasting, dispositivo e custo de operações vetorizadas.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

PROVENIÊNCIA E PRÉ-REQUISITOS

- Material histórico de apoio: Unidade 2 — Preliminares I, páginas 43–80 do PDF consolidado.
-

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Tensores generalizam escalares, vetores e matrizes. As dimensões do batch, dos atributos, dos canais e do tempo dependem do domínio. A multiplicação matricial é distinta do produto elemento a elemento.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: (B,d) @ (d,h) = (B,h)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Multiplicação elemento a elemento, produto matricial e *broadcasting* têm semânticas distintas. A ordem dos eixos importa para imagens: no formato channels_last, um lote RGB possui forma (B, H, W, 3). Os exemplos seguintes usam apenas NumPy, garantindo execução sem TensorFlow.
Análise e exercícios

Interpretação: eixo do lote (axis=0) não é eixo de canal (axis=-1). O NumPy informa incompatibilidade de forma; não se deve silenciar isso com um reshape arbitrário. Atividade: explique como as formas mudariam em channels_first e quando seria necessário transpor os eixos.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
X=np.arange(12,dtype='float32').reshape(4,3)
W=np.array([[1.,0.],[0.,1.],[-1.,2.]],dtype='float32')
Z=X@W
assert Z.shape==(4,2)
print('X',X.shape,'W',W.shape,'XW',Z.shape,'dtype',Z.dtype)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de

modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório reproduzível — broadcasting, dtype e formas

Multiplicação elemento a elemento, produto matricial e *broadcasting* têm semânticas distintas. A ordem dos eixos importa para imagens: no formato channels_last, um lote RGB possui forma (B, H, W, 3). Os exemplos seguintes usam apenas NumPy, garantindo execução sem TensorFlow.
Análise e exercícios

Interpretação: eixo do lote (axis=0) não é eixo de canal (axis=-1). O NumPy informa incompatibilidade de forma; não se deve silenciar isso com um reshape arbitrário. Atividade: explique como as formas mudariam em channels_first e quando seria necessário transpor os eixos.

```
import numpy as np

lote = np.arange(2 * 3 * 4 * 3, dtype=np.float32).reshape(2, 3, 4, 3)
media_por_canal = lote.mean(axis=(0, 1, 2))
centrado = lote - media_por_canal  # broadcasting: (3,) -> (2,3,4,3)
assert centrado.shape == (2, 3, 4, 3)
assert np.allclose(centrado.mean(axis=(0, 1, 2)), 0, atol=1e-6)

X = np.arange(12, dtype=np.float32).reshape(4, 3)
W = np.arange(6, dtype=np.float32).reshape(3, 2)
Z = X @ W
assert Z.shape == (4, 2)
print('lote:', lote.shape, 'média por canal:', media_por_canal)
print('produto matricial:', Z.shape, 'dtype:', Z.dtype)

try:
    _ = X @ np.ones((4, 2), dtype=np.float32)
except ValueError as erro:
    print('Shape incompatível detectado corretamente:', type(erro).__name__)
else:
    raise AssertionError('Era esperado erro por dimensões incompatíveis.')
```

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Qual o resultado dimensional de 32 exemplos com 16 entradas e 8 neurônios?

Resposta: Uma saída de shape (32, 8), com matriz de pesos (16, 8) e bias (8,).

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide